

第二章：数据质量与准备 (Data Quality & Preparation)

1. DQR (数据质量报告) vs DQP (数据质量计划)

- Data Quality Report (DQR): 负责"诊断"。列出所有特征的统计数据 (如 % Missing, Cardinality, Min, Max), 并通过图表 (Histograms, Boxplots) 展示分布和异常点。
- Data Quality Plan (DQP): 负责"开药方"。针对 DQR 发现的每个问题, 提出具体的处理动作 (Action) 并执行。
 - 建议的执行顺序: 先处理结构性问题 (如 Drop 无用特征、处理严重缺失的行/列), 然后再做精细化处理 (Imputation 缺失值插补、Clamping 异常值截断)。

1.1 Motor Insurance DQR 实际案例 (Lab3 + Lab4)

数据集: MotorInsuranceFraudClaimABTFull.csv — 500行 × 14列

数值特征摘要 (DQR核心产出):

Feature	%缺失	基数	关键问题
Income of Policy Holder	0%	171	中位数=0, 零值占比高
Claim Amount	0%	493	**存在负值 -99999** (疑似录入错误)
Total Claimed	0%	235	最大值 729792, 长尾分布
Num Claims	0%	7	最大值 56, 离散明显
Num Soft Tissue	2%	5	**存在少量缺失**
Claim Amount Received	0%	329	长尾分布, 存在大额值

类别特征摘要:

Feature	%缺失	基数	关键问题
ID	0%	500	**每行唯一**, 不作为特征
Insurance Type	0%	1	**常量列** (全为"CI"), 建议删除
Marital Status	**66%**	3	**缺失极严重**, 需重点处理
Fraud Flag	0%	2	目标变量, 类别不平衡 (332:168)

1.2 DQP 实际动作案例 (来自 Lab4)

针对 DQR 发现的问题, DQP 的执行动作:

- **InsuranceType**: 常量列 → DROP
- **MaritalStatus**: 缺失 66% → 删除列或标记为 Unknown
- **ClaimAmount 负值 -99999**: 用 ClaimAmountReceived 字段替换修复

- ****ClaimAmount 高位异常值: 按 95 分位数做 Clamping (截断) ****
- ****NumSoftTissue 缺失 (2%)*:** 用 0 填补 (基于业务逻辑)
- **备份原始数据:** `df_raw = df.copy()` 确保可回溯

```
# DQP Lab4 Notebook

# Step 1:

df_raw = df.copy()


# Step 2:

df = df.drop(['InsuranceType', 'MaritalStatus', 'ID'], axis=1)


# Step 3: - ClaimAmountReceived ClaimAmount

mask_negative = df['ClaimAmount'] < 0

df.loc[mask_negative, 'ClaimAmount'] = df.loc[mask_negative, 'ClaimAmountReceived']


# Step 4: Clamping - 95

upper_quantile = df['ClaimAmount'].quantile(0.95)

df['ClaimAmount'] = df['ClaimAmount'].clip(upper=upper_quantile)


# Step 5: NumSoftTissue

df['NumSoftTissue'] = df['NumSoftTissue'].fillna(0)


# Step 6:

print("Cleaned dataset shape:", df.shape)

print("Missing values after cleaning:\n", df.isnull().sum())
```

2. 缺失值处理 (Handling Missing Values)

处理缺失值需根据缺失原因 (Random vs Systematic) 采取不同策略。

2.1 常见策略与 Python 代码

```
import pandas as pd

import numpy as np


# 1: (Drop Feature)

df = df.drop('Feature_with_90pct_missing', axis=1)


# 2: (Drop Rows - )

df = df.dropna(subset=['Critical_Feature'])


# 3: // (Imputation)

# mean median

df['Numeric_Col'] = df['Numeric_Col'].fillna(df['Numeric_Col'].median())

# (mode)

df['Cat_Col'] = df['Cat_Col'].fillna(df['Cat_Col'].mode()[0])


# 4: (Logical Imputation)

# 'Percentage' 0 0

df.loc[df['Percentage'] == 0, 'Related_Feature'] = df.loc[df['Percentage'] == 0,
'Related_Feature'].fillna(0)
```

3. 异常值处理 (Handling Outliers)

在数据分析中，离群点不一定是错误的 (Invalid data)，它们可能是极端但真实的业务数据 (Valid extreme data)。直接删除 (Delete) 会导致信息丢失，截断 (Clamping) 是更好的方法。

3.1 异常值判定逻辑 (IQR Method)

推导与计算过程:

- 计算第一四分位数 $Q1$ (25th percentile) 和第三四分位数 $Q3$ (75th percentile)。
- 计算四分位距 $IQR = Q3 - Q1$ 。
- 定义下界 (Lower Bound) $= Q1 - 1.5 \times IQR$
- 定义上界 (Upper Bound) $= Q3 + 1.5 \times IQR$

任何超出 $[Lower, Upper]$ 区间的值都被视为异常值。

3.2 Clamping (截断) 的 Python 实现

```
# 1.  Q1, Q3  IQR

Q1 = df['Salary'].quantile(0.25)

Q3 = df['Salary'].quantile(0.75)

IQR = Q3 - Q1


# 2.

lower_bound = Q1 - 1.5 * IQR

upper_bound = Q3 + 1.5 * IQR


# 3. Clamping

df['Salary_Clamped'] = df['Salary'].clip(lower=lower_bound, upper=upper_bound)
```

4. 特征缩放与归一化 (Feature Scaling)

机器学习算法 (如 K-NN, 神经网络) 对数值特征的绝对大小(Scale)非常敏感。需要将它们缩放到相同的量级。

4.1 Min-Max Normalisation (极差归一化)

将数据缩放到指定区间 $[a, b]$ (通常是 $[0, 1]$ 或 $[-1, 1]$)。数学公式推导: $x_{\text{new}} = \frac{x - \min(x)}{\max(x) - \min(x)} \times (b - a) + a$

****致命弱点**:** 对异常值极度敏感 (Extremely sensitive to outliers)。如果存在一个巨大的 max 值, 大部分正常数据会被挤压在一个极其狭窄的微小区间内。

是一种将数据线性映射到固定区间（通常为 [0,1]）的方法。它适合量纲不同且异常值较少的数据，能够保留数据的相对大小关系，但对异常值较敏感。

4.2 Z-Score Standardisation (标准化)

将数据转化为均值 $\mu = 0$ ，标准差 $\sigma = 1$ 的标准正态分布。数学公式: $x_{\text{new}} = \frac{x - \mu}{\sigma}$

****优点**:** 相比 Min-Max，它对异常值的鲁棒性更好 (More robust to outliers)，因为它不强行限定数据的上下界。

Z-Score Standardisation 是将数据减去均值后再除以标准差，使处理后的数据均值为 0、标准差为 1。它反映每个数据点距离均值有多少个标准差，常用于不同量纲数据的比较和模型预处理

4.3 异常值对缩放的影响 (Lab5 核心实验)

关键实验对比 — 有异常值 vs 无异常值时，两种缩放方法的表现完全不同：

```
# =====

# : [1, 2, 3, 4, 5]

# Min-Max → [0.00, 0.25, 0.50, 0.75, 1.00]

# Z-Score → [-1.41, -0.71, 0.00, 0.71, 1.41]

# ===== 100 =====

# : [1, 2, 3, 4, 5, 100]

# Min-Max → [0.00, 0.01, 0.02, 0.03, 0.04, 1.00] 5 [0, 0.04]

# Z-Score → [-0.49, -0.47, -0.44, -0.42, -0.39, 2.21]
```

Lab5 Basketball 数据实验结论

Lab5 使用 BasketballTeam.csv 数据，先对 Height/Weight/SponsorshipEarnings 做正常缩放，然后人为加入异常值后重新缩放：

- ****Min-Max**:** 加入异常值后，正常数据的直方图被压扁到一个极窄区间，失去区分度

- ****Z-Score**:** 加入异常值后，正常数据的分布形状基本保持不变，只是整体偏移

****结论**:** 如果数据中存在异常值且你无法清洗它们，****优先选择 Z-Score Standardisation****。

4.4 Python Scikit-Learn 实现

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Min-Max Normalization ( [0, 1])

min_max_scaler = MinMaxScaler()

df['Feature_MinMax'] = min_max_scaler.fit_transform(df[['Feature']])


# Z-Score Standardization

std_scaler = StandardScaler()

df['Feature_Std'] = std_scaler.fit_transform(df[['Feature']])
```

5. 分箱 (Binning) 与数据划分 (Sampling)

5.1 分箱 (Binning)

将连续变量离散化为有序类别 (Ordinal categories)。有助于处理非线性关系并降低异常值影响。

```
# (Equal-frequency/Quantile binning):

df['Age_Binned'] = pd.qcut(df['Age'], q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
```

5.2 随机抽样 vs 分层抽样 (Lab5 实验对比)

Simple Random Sampling (简单随机抽样):

不区分类别，直接随机抽取。在类别不平衡时极易导致某一类在训练集/测试集中完全消失。

Stratified Sampling (分层抽样): 先按类别分层，再在每一层内分别随机抽样，确保划分后各类别比例与原始数据一致。

Lab5 Basketball数据 — 不平衡标签下的抽样对比

假设构造了一个标签严重不平衡的数据 (正类 90%, 负类 10%):

****随机抽样结果**:** 测试集中负类样本可能会减少甚至消失

...

训练集: 正类 63, 负类 7 (90% : 10%)

测试集: 正类 27, 负类 0 (100% : 0%) 测试集评估完全无效!

...

****分层抽样结果**:** 训练集和测试集的类别比例完全一致

...

训练集: 正类 63, 负类 7 (90% : 10%)

测试集: 正类 27, 负类 3 (90% : 10%) 测试集能真实反映模型在少数类上的表现

...

5.3 数据泄露防范 (Data Leakage)

核心原则

****必须在任何基于整体分布的转换（如标准化、归一化、均值插补、One-Hot编码）之前进行训练集/测试集的划分**！** 否则测试集的信息会泄露到训练过程中，导致虚假的高分。

```
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, MinMaxScaler

X = df.drop('Target', axis=1)
y = df['Target']

# : fit_transform transform

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42, stratify=y)

#

scaler = StandardScaler() # MinMaxScaler()

X_train_scaled = scaler.fit_transform(X_train) # fit + transform:  $\mu$ ,  $\sigma$ 

X_test_scaled = scaler.transform(X_test) # transform:  $\mu$ ,  $\sigma$ 

#  $\Delta$  : /  $\rightarrow$ 

# : scaler.fit_transform(X)  $\rightarrow$  train_test_split(...)
```

为什么测试集只能 transform，不能 fit_transform？

- ****fit_transform(X_train)****: 用训练集计算均值 μ 和标准差 σ ，然后转换训练集
- ****transform(X_test)****: 用**训练集的 μ 和 σ** 转换测试集，而不是重新计算
- 如果对测试集也 fit_transform，测试集的分布会影响缩放参数，导致****数据泄露**** (Data Leakage)